

This document reflects the state of the codebase as of July 21, 2026. All findings cite actual source code or configuration. Items marked "not implemented" were not found in the codebase — they are not described as intended behaviors.

Contents

- 1. Data Flow & PHI Boundaries
- 2. Integrations & External Systems
- 3. Access Control & Audit
- 4. Hosting & Compliance Posture
- 5. Open Risks / Not Yet Built

1. Data Flow & PHI Boundaries

PHI Entry Points

PHI enters the system through three distinct paths:

- Manual clinical entry via `/api/assessment` (`server/routes.ts:652`). Clinicians submit patient age, pregnancy status, HLA-B*5701 status, viral load, CD4 count, eGFR, hepatic score, current HIV regimen, proposed regimen, and concomitant medications. Patient initials are used, not full legal name — but the combination of clinical parameters meets HIPAA's expert-determination threshold for PHI.
- Manual retention entry via `/api/retention/patients` (`server/routes.ts:1494`). Staff enter patient initials, two phone numbers, email address, city, zip code, and insurance BIN/PCN/RxGroup/ID. This is unambiguous PHI. Bulk equivalents enter via CSV upload (`/api/retention/import`) and an SSRS data sync script.
- Inbound JotForm webhook (`/api/webhooks/jotform`). Triggered when a patient completes a Patient Service Agreement. The server fetches the completed PDF from JotForm's API (`server/lib/faxService.ts`). The PDF may contain the patient's full legal name and signature — the highest-fidelity PHI ingestion point in the system.

PHI at Rest — Storage Locations

Store	Contents	PHI Present
PostgreSQL retention_patients	Initials, phones, email, zip, insurance BIN/PCN/RxGrp/ID — full JSONB record	YES
PostgreSQL client_store	Server-side sync of koheez_assessments localStorage key — contains clinical parameters	YES
PostgreSQL session	Active session tokens via connect-pg-simple	Session identity only
PostgreSQL audit_logs	Actor email, name, role, action, resource UUID, timestamp	Actor PII; no patient PHI by design
PostgreSQL cqi_meetings	Staff names and SVG-format digital signatures	Staff PII, not patient PHI
Browser localStorage koheez_assessments	Clinical parameters written on every assessment save	YES — persists after session ends
JotForm servers (third-party)	Patient Service Agreement PDFs — may include full legal name	YES — third-party hold
In-memory (transient)	Request-scoped processing only	Cleared after each request

PHI in Transit — External System Map

Every external system this application transmits PHI to or receives PHI from:

Destination	PHI Transmitted	Direction	Auth Method
OpenAI (gpt-4o) <code>server/routes.ts:806</code>	Patient age, clinical labs, HIV regimen, concomitant meds — full clinical profile	Outbound Real-time	API key — standard tier, NO BAA
OpenEvidence <code>server/lib/openEvidence.ts</code>	Same clinical profile (primary path; OpenAI is fallback)	Outbound Real-time	API key + optional session cookie
Liverpool HIV DDI <code>server/lib/liverpoolDDI.ts</code>	Drug IDs resolved from patient's medication list	Outbound Real-time	X-API-Key header
Salesforce <code>server/lib/salesforceClient.ts</code>	Patient phone number (Contact lookup), initials, outreach event details	Outbound Real-time	OAuth2 password grant (deprecated)

Clerkchat server/lib/clerkchatClient.ts	Patient phone number, SMS message body	Outbound Real-time	API key (Bearer)
Microsoft Graph / Outlook server/lib/outlookClient.ts	Recipient email address, message body marked "Confidential"	Outbound Real-time	Static Bearer token — no refresh
JotForm server/lib/faxService.ts	Submission ID sent; full Patient Service Agreement PDF received	Inbound then out	API key as URL query param (logged)
sFax server/lib/faxService.ts	Binary PDF content (Patient Service Agreement)	Outbound	Username + API key as form-data
NIH RxTerms (public)	Partial drug name strings (autocomplete)	Outbound	None — public API
ADP Workforce Now server/lib/adpClient.ts	Worker OIDs and site codes only — no patient PHI	Outbound Nightly batch	OAuth2 + mTLS (X.509)
SSRS (inbound) scripts/ssrs-sync/	Patient initials, phone, issue type	Inbound Batch	Static IMPORT_API_KEY header

Environments Touching PHI

The application has a single Dockerfile targeting Aptible (Dockerfile:1). There is no staging environment defined in the codebase. Both the development environment (Replit) and production (Aptible) reference the same DATABASE_URL environment variable — there is no separate dev database configured in any file in the repository. If this single DATABASE_URL points to the production Neon instance (the default configuration pattern), the Replit development environment has live PHI access. Replit is not a HIPAA-eligible hosting provider and does not offer a BAA.

PHI Boundary Gaps

- Clinical data (age, labs, HIV regimen) is transmitted to the standard OpenAI API without a BAA. This is the most significant PHI boundary violation in the system. Source: server/routes.ts:806, server/routes.ts:941.
- OpenEvidence code comment claims "HIPAA-compliant infrastructure" (server/lib/openEvidence.ts:14) but this is a developer comment, not a contract. No BAA evidence in codebase.
- JotForm API key is passed as a URL query parameter (?apiKey=...), meaning it appears in server access logs and proxy logs. Source: server/lib/faxService.ts.
- The JotForm webhook endpoint (/api/webhooks/jotform) accepts POST requests from any caller with no authentication. An attacker can submit arbitrary submissionIDs, causing the server to fetch arbitrary JotForm PDFs.

2. Integrations & External Systems

Integration Inventory

System	Data Moving	Direction	Trigger	Auth	Known Issues
OpenAI (gpt-4o)	Clinical assessment params, note content	Outbound	Real-time per request	API key (env: OPENAI_API_KEY)	No BAA. Rate limit returns 402 to client with no retry logic.
OpenEvidence	Clinical profile, ARV details	Outbound	Real-time; OpenAI fallback	API key + session cookie	Session cookie credential implies browser-session auth, not machine credential.
Liverpool HIV DDI	Drug ID arrays	Outbound	Real-time per assessment	X-API-Key header	No caching — every assessment makes at least one outbound call.
Salesforce	Patient phone (lookup), initials, outreach events	Outbound	Real-time per event	OAuth2 password grant (env: SF_USERNAME, SF_PASSWORD, SF_SECURITY_TOKEN)	Password grant is deprecated by Salesforce. No refresh token. Credential rotation breaks integration silently.
Clerkchat	Patient phone, SMS body	Outbound	Real-time per outreach event	API key (Bearer) (env: CLERKCHAT_API_KEY)	None identified.
Microsoft Graph (Outlook)	Recipient email, message body	Outbound	Real-time per outreach event	Static Bearer token (env: OUTLOOK_ACCESS_TOKEN)	No refresh mechanism. Token expiry causes silent failure with no error recovery path in code.
JotForm sFax	Patient Service Agreement PDF	Inbound then outbound	Event-driven webhook	JotForm: API key as query param sFax: form-data creds	Webhook has no inbound auth. API key exposed in URL (appears in logs).
ADP Workforce Now	Worker OIDs, site codes (no patient PHI)	Outbound	Nightly batch cron (0 2 * * *)	OAuth2 client creds + mTLS X.509 (env: ADP_CLIENT_ID, ADP_SSL_CERT)	Most secure integration. No issues identified.
SSRS (SQL Server)	Patient initials, phone, issue type	Inbound	Batch via sync script	Static shared key (env: IMPORT_API_KEY)	No per-caller identity. Shared key compromise affects all callers.
NIH RxTerms	Partial drug name strings (autocomplete)	Outbound	Real-time (client-side)	None — public API	No PHI transmitted. No issues identified.
Microsoft Teams / Azure AD	Clinician identity tokens only — no data sync	Inbound (SSO)	On login	Azure AD JWT validation (env: AAD_APP_CLIENT_ID)	Only users pre-provisioned in hardcoded PROFILE_MAP can sign in via Teams.

Secret Storage

All secrets are stored as environment variables on the host (Replit Secrets in dev, Aptible config vars in production). There is no secrets manager (Vault, AWS Secrets Manager, etc.) in use. No secrets are committed to the repository. The mTLS certificate and key for ADP are stored as environment variables with literal `\n` sequences (server/lib/adpClient.ts).

3. Access Control & Audit

Role Hierarchy

Defined in `client/src/lib/userProfile.ts`. Role assignment is done by matching the authenticated email address against a hardcoded `PROFILE_MAP` in source code. Adding or removing a user requires a code deployment.

Role	Scope	PHI Access	Notable Permissions
Chief Pharmacy Officer (CPO)	All regions and sites	Full	Only role that can view HIPAA audit logs (<code>/api/audit-log</code>)
Regional Pharmacy Director (RPD)	Assigned region	Retention patients in region	Can approve site schedule submissions
Pharmacy Director (PD)	Assigned site(s)	Retention patients at site	Manages site hours, staff schedule, ACHC workbook
pharmacist_1, pharmacist_2	Assigned site	Clinical assessments, retention patients at site	Can run drug interaction checks, generate clinical notes
data_entry_tech, pv2_tech, delivery_tech	Assigned site	Read-only on controlled inventory	No clinical assessment access

Authentication Implementation

- Standard login: `username/password` checked with strict equality — `u.password === password` (`server/routes.ts:439`). No hashing is applied. `bcrypt/Argon2` are not installed.
- All named staff accounts share a single plaintext password ("AHF1") hardcoded in `DEMO_USERS` array (`server/routes.ts:73-169`). This array contains real employee email addresses from `aidshealth.org` and `ahfrx.org` domains.
- `SESSION_SECRET` defaults to the literal string "koheez-dev-secret" when the environment variable is absent (`server/index.ts:59`). If unset in production, session tokens are trivially forgeable.
- Teams SSO: Validates Azure AD JWTs. Only emails present in hardcoded `PROFILE_MAP` are permitted. This is a separate, stronger auth path — but does not cover standard password logins.
- No CSRF middleware is present in the codebase. Cookies use `SameSite=None` in production (required for Teams iframe embedding), which increases the CSRF attack surface.

Audit Log — What Is Implemented

An audit log is implemented and persisted in PostgreSQL (`audit_logs` table, `shared/schema.ts:281`). Fields captured: actor email, name, role, action string, resource UUID, HTTP method, path, timestamp.

Actions that ARE audited (calls to `recordAccess` in `server/routes.ts`):

- Teams SSO login events (line 499)
- Assessment generation (line 652)
- Clinical note generation (line 892)
- CQI meeting list, read, save, sign (lines 1339–1444)
- Retention patient read, create, update, delete (lines 1451–1534)

Actions that ARE NOT audited:

- Standard `username/password` logins — the login route does not call `recordAccess`. If shared credentials are used, there is no record of individual access.
- Bulk patient CSV/SSRS imports (`/api/retention/import`) — large-scale PHI ingestion events are invisible in the audit log.
- `/api/client-store` reads and writes — this endpoint backs up clinical assessments (PHI) from `localStorage` to the database with no audit entry.
- Controlled inventory reads and writes.
- Any user management operations (none exist in the UI — user management requires code changes).

Audit log retention policy: not implemented. No TTL, export function, or tamper-evident mechanism. HIPAA requires six years of audit log retention.

4. Hosting & Compliance Posture

Deployment Architecture

Production target: Aptible (HIPAA-eligible PaaS). Confirmed by Dockerfile:1. Aptible offers a BAA — whether one is signed is not determinable from the codebase.

Development environment: Replit. Replit is not a HIPAA-eligible hosting provider and does not offer a BAA. The codebase — including all hardcoded staff credentials and the DATABASE_URL environment variable — is accessible within the Replit workspace.

Database: Neon serverless PostgreSQL. Neon offers a BAA under Business/Enterprise plans. Whether one is in place is not determinable from the codebase.

BAA Status by Vendor

Vendor	BAA Required?	Evidence in Codebase
Aptible (hosting)	YES	Dockerfile confirms Aptible target. BAA existence cannot be confirmed from code.
Neon (database)	YES	DATABASE_URL connection used. BAA existence cannot be confirmed from code.
OpenAI	YES — PHI transmitted	Standard API endpoint used (api.openai.com). No evidence of HIPAA enterprise agreement. This is a confirmed gap.
OpenEvidence	YES — PHI transmitted	Code comment in openEvidence.ts:14 says "HIPAA-compliant infrastructure." This is a comment, not a contract.
Salesforce	YES — PHI transmitted	No evidence of BAA in codebase.
Clerkchat	YES — patient phones transmitted	No evidence of BAA in codebase.
Microsoft (Graph/Teams)	YES	Microsoft 365 BAA exists for Teams — but does not automatically cover third-party apps. AHF's Microsoft tenant configuration is not visible in codebase.
JotForm	YES — PDF with patient data	No evidence of BAA in codebase.
sFax	YES — PHI fax content	No evidence of BAA in codebase.

Session & Cookie Security

httpOnly: true. secure: true in production. SameSite=None in production (Teams iframe requirement). No CSRF protection middleware present in codebase.

Data Retention

Not implemented. No TTL, archival, or deletion policy is enforced in code for retention_patients, client_store, or audit_logs tables. PHI accumulates indefinitely.

5. Open Risks / Not Yet Built

The following gaps would be cited in a formal HIPAA or informatics security review. Presented without optimistic framing.

CRITICAL

1

CRITICAL

PHI transmitted to OpenAI without a BAA. Clinical assessment parameters (age, labs, HIV regimen) are sent to `api.openai.com` using the standard commercial API tier. No HIPAA-eligible agreement or enterprise endpoint is configured. Source: `server/routes.ts:806, 941`.

2

CRITICAL

Plaintext password storage and comparison. All user credentials are stored as plaintext strings and compared with `===`. No hashing algorithm (bcrypt, Argon2, etc.) is applied anywhere in the codebase. Source: `server/routes.ts:439`.

3

CRITICAL

All users share a single hardcoded password. Every named staff account (30+ real employee email addresses) uses the password "AHF1." Individual user accountability is impossible with shared credentials. Source: `server/routes.ts:73–169`.

4

CRITICAL

`SESSION_SECRET` defaults to a known public string. If the `SESSION_SECRET` environment variable is not set in production, session tokens are signed with "koheez-dev-secret" — a string visible in public source code. Source: `server/index.ts:59`.

5

CRITICAL

JotForm webhook is unauthenticated. `/api/webhooks/jotform` accepts POST from any caller. Attackers can supply arbitrary submissionIDs, causing the server to fetch arbitrary JotForm PDFs on their behalf.

HIGH

6

HIGH

Password-based logins are not audited. The login route does not call `recordAccess`. If a shared credential ("AHF1") is used by multiple people, there is no record of individual access in the audit log.

7

HIGH

Bulk patient import is not audited. `/api/retention/import` (CSV and SSRS inbound) has no `recordAccess` call. Large-scale PHI ingestion is invisible to the audit trail.

8

HIGH

`/api/client-store` writes are not audited. This endpoint syncs clinical assessments (PHI) from `localStorage` to the database on a debounced write cycle with no audit entry.

9

HIGH

User management requires a code deployment. There is no admin interface, no deprovisioning workflow, and no way to immediately revoke access for a terminated employee without a code push and redeploy.

10

HIGH

Static Outlook Bearer token. `OUTLOOK_ACCESS_TOKEN` is a non-refreshing credential stored as an environment variable. Expiry causes silent failure of all patient outreach emails. No error recovery path exists in code. Source: `server/lib/outlookClient.ts`.

11

HIGH

No CSRF protection. No CSRF token middleware is present. `SameSite=None` cookies in production increase the attack surface for cross-site request forgery.

12

HIGH

Replit development environment likely has production database access. Replit is not HIPAA-eligible and does not offer a BAA. If DATABASE_URL in Replit Secrets points to the production Neon instance — the default single-database configuration — PHI is accessible from a non-covered environment.

MEDIUM

13

MED

No data retention or deletion policy. PHI in retention_patients and client_store accumulates indefinitely. HIPAA minimum necessary principle and state law deletion requirements are not enforced in code.

14

MED

Audit log has no retention, export, or integrity controls. HIPAA requires six-year audit log retention. No export function, no tamper-evident mechanism, and no automated retention policy exist in the codebase.

15

MED

JotForm API key passed as a URL query parameter. The key appears in server access logs, proxy logs, and any HTTP request monitoring. Source: server/lib/faxService.ts.

16

MED

Clinical assessments in browser localStorage are unencrypted. koheez_assessments persists after session ends. On shared or unmanaged workstations, this data is readable by any script on the same origin.

17

MED

Salesforce uses deprecated OAuth2 password grant. This flow transmits credentials directly and does not support refresh tokens. Salesforce has been disabling password grant for orgs since 2022 without explicit re-enablement. Source: server/lib/salesforceClient.ts.

18

MED

No minimum password length, complexity, or rotation policy is enforced. Signup validation applies no complexity rules. All 30+ existing staff accounts use a six-character shared password with no expiry.